

# プログラミングI 演習

## Computer Programming I: Exercises

---

### 第9回 ポインタと配列

#### 担当

A-P1 辻 愛里

A-P2 北 直樹

A-P3 清水 昭伸

1

## 本日の講義

### 本日の目標

- ポインタと配列の関係を覚えよう
- 配列をポインタで表現してみよう

### 本日の資料

---

1. 前回のおさらい
2. ポインタと配列
3. ポインタと配列(上級者向け)
4. 演習課題

# 1. 前回のおさらい

## 変数の局所性

変数「x」という名前が使われている場所

↓main関数とSub関数で同じ名の変数がある

```
double Sub(double x, double y)
{
    x = x - y;    変数「x」を使って計算
    return x;
}

int main(void)
{
    double x=5.0, y=1.0;
    printf("%f-%f = %f\n", x, y, Sub(x,y));
    printf("x=%f\n", x);
    return 0;
}
```

関数内でxの値を変更してしまっているが、呼び出し側(main)に影響はないのだろうか？



引数や関数内で宣言した変数は関数毎に独立に存在している

「x=5.000000」と表示される

3

# 1. 前回のおさらい

## 配列を引数とする関数

```
void risi(int List[100], double Interest)
{
    int i;
    for(i=0; i<100; i++)
        List[i] = List[i] * (1.0 + Interest);
}
```

```
int main(void)
{
    int Account[100];
    .....
    risi(Account, 0.003);
    ..
}
```

引数で受け取った配列Listを更新するだけでmainのAccountも更新

関数に渡すのは配列名(同名のポインタ)  
(→つまりポインタ渡し)

int Account [100];

| 添え字   | 残高         |
|-------|------------|
| 0     | 12345678   |
| 1     | 2345678    |
| 2     | 1234567    |
| ..... | .....      |
| 97    | 345678     |
| 98    | 5678       |
| 99    | 1234567890 |

4

# 1. 前回のおさらい

## 演習8-4 ベクトル演算

- [1] 整数 $k$ と一次元配列 $a$ (大きさが $n$ のベクトル. 各成分は整数)を引数とし, 配列 $a$ をスカラー倍( $k$ 倍)する関数を定義せよ

例

```
void Scalar(int a[10], int k)
```

- [2] 一次元配列 $a$ と一次元配列 $b$ (どちらも大きさが $n$ のベクトル. 各成分は整数)を引数とし,  $a+b$ を行う関数を定義せよ

例

```
void AddVector(int a[10], int b[10])
```

5

# 1. 前回のおさらい

## 演習8-4[1] 解答例

```
#include <stdio.h>
#define N 10

void Scalar(int a[N], int k){
    int i;
    for(i=0; i<N; i++) {a[i] = k * a[i];}
}

int main(void){
    int i;
    int a[N], k=3;
    for(i=0; i<N; i++) {a[i] = i;}
    Scalar(a, k);
    for(i=0; i<N; i++) {printf("a[%d]は%d\n", i, a[i]);}
    return 0;
}
```

問題は「関数を定義せよ」だが、  
定義できたか否かを確認するために  
main関数も作成する必要がある

6

# 1. 前回のおさらい

## 演習8-4[2] 解答例

```
#include <stdio.h>
#define N 10

void AddVector(int a[N], int b[N]){
    int i;
    for(i=0; i<N; i++) {a[i] = a[i] + b[i];}
}

int main(void){
    int i;
    int a[N], b[N];
    for(i=0; i<N; i++) {a[i] = i;}
    for(i=0; i<N; i++) {b[i] = i;}
    AddVector(a, b);
    for(i=0; i<N; i++) {printf("a[%d]は%d\n", i, a[i]);}
    return 0;
}
```

- ・引数が配列2つの場合、片方の配列に答を上書き
- ・元のデータが失われるのを避けるためには、答を入れる配列を3つめの引数として用意する必要あり

7

# 1. 前回のおさらい

## 発展8-1 行列演算

[1] 二次元配列 $a$ と二次元配列 $b$ (どちらも $m \times n$ のサイズの行列、各成分は整数)を引数とし、 $a+b$ を行う関数を定義せよ

例

```
void AddMatrix(int a[5][10], int b[5][10])
```

[2] 二次元配列 $a$ と二次元配列 $b$ (どちらも $n \times n$ のサイズの行列、各成分は整数)を引数とし、 $a \times b$ を行う関数を定義せよ

例

```
void MultiMatrix(int a[5][5], int b[5][5])
```

8

# 1. 前回のおさらい

## 発展8-1[1] 解答例

```
#include <stdio.h>
#define M 5
#define N 10
void AddMatrix(int a[M][N], int b[M][N]){
    int i,j;
    for(i=0; i<M; i++)
        for(j=0; j<N; j++)
            {a[i][j] = a[i][j] + b[i][j];}
}
int main(void){
    int i,j;
    int a[M][N], b[M][N];
    for(i=0; i<M; i++) for(j=0; j<N; j++) {a[i][j] = i*j;}
    for(i=0; i<M; i++) for(j=0; j<N; j++) {b[i][j] = 1;}
    AddMatrix(a, b);
    for(i=0; i<M; i++){
        for(j=0; j<N; j++){
            printf("%3d", a[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

配列aには各要素に*i\*j*の値を入れて初期化  
配列bにはすべての要素に1を入れて初期化  
関数でa+bを実行し、結果を表示

$$a = \begin{pmatrix} 0 & 0 & 0 & \dots & 0 \\ 0 & 1 & 2 & \dots & 9 \\ 0 & 2 & 4 & \dots & 18 \\ 0 & 3 & 6 & \dots & 27 \\ 0 & 4 & 8 & \dots & 36 \end{pmatrix}$$

9

# 1. 前回のおさらい

## 発展8-1[2] 解答例

```
#include <stdio.h>
#define N 3
void MultiMatrix(int a[N][N], int b[N][N]){
    int i,j,k;
    int ans[N][N]={0}; /* すべての成分に0が入る */
    for(i=0; i<N; i++)
        for(j=0; j<N; j++)
            for(k=0; k<N; k++)
                ans[i][j] = ans[i][j] + a[i][k] * b[k][j];
    for(i=0; i<N; i++)
        for(j=0; j<N; j++)
            a[i][j] = ans[i][j];
}
int main(void){
    int i,j;
    int a[N][N], b[N][N];
    for(i=0; i<N; i++) for(j=0; j<N; j++) a[i][j] = i*j;
    for(i=0; i<N; i++) for(j=0; j<N; j++) b[i][j] = 2;
    MultiMatrix(a, b);
    for(i=0; i<N; i++){
        for(j=0; j<N; j++){
            printf("%3d", a[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

行列の積の計算のための三重ループ

$$ans_{ij} = \sum_{k=1}^3 a_{ik} b_{kj}$$

$$\begin{pmatrix} ans_{11} & ans_{12} & ans_{13} \\ ans_{21} & ans_{22} & ans_{23} \\ ans_{31} & ans_{32} & ans_{33} \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix} \begin{pmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{pmatrix}$$

$ans_{ij} = \sum_{k=1}^3 a_{ik} b_{kj} \rightarrow [i][j]$ 要素は*k*に関する和により計算

演習8-4や発展8-1のように、直接配列aに代入せず、一度配列ansに入れた後で配列aに代入している理由は？

$$a = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 1 & 2 \\ 0 & 2 & 4 \end{pmatrix}, \quad b = \begin{pmatrix} 2 & 2 & 2 \\ 2 & 2 & 2 \\ 2 & 2 & 2 \end{pmatrix}$$
$$ab = \begin{pmatrix} 0 & 0 & 0 \\ 6 & 6 & 6 \\ 12 & 12 & 12 \end{pmatrix}$$

10

# 1. 前回のおさらい

## 発展8-2 血液型調査プログラム

(※難易度はかなり高めです. 完成できなくても, 関数の機能を考えたり, 大まかなプログラムを考えるだけでも力になります)

データ保存用の配列を使う

```
int BloodTypeList[100];
```

| 添え字   | 血液型   |
|-------|-------|
| 0     | 2     |
| 1     | 1     |
| 2     | 3     |
| ..... | ..... |
| 97    | 0     |
| 98    | 4     |
| 99    | 1     |

A型=1、B型=2、O型=3、AB型=4  
不明&未調査=0

### 要求される機能

1. 情報の登録/修正
2. 指定した型の人の番号を一覧表示
3. 統計(何型が何人いる)情報の表示



それぞれの処理を行う関数を実装しよう

プログラム例は補足資料  
「BloodType+.c」

11

## 2. ポインタ

C言語の大事な概念で, 身に着けると出来ることが増えます

- ポインタ/ポインタ変数: 「アドレス」を保存するための変数
- 関数に配列を引数として渡すとき, 「値渡し」「ポインタ渡し」の違いを知っておく必要がある

### ポインタを使うことの代表的な利点

- 関数から戻値を(実質)複数受け取れる
- 関数に複数のデータを(=配列やファイルを)渡せる
- 動的メモリ割当てが可能になる

### 本演習で学ぶこと

- ポインタ変数の宣言・読み書きのための文法
- ポインタ渡しの文法
- 「値渡し」と「ポインタ渡し」の違い

12

## 2. ポインタと配列

### 配列を引数とする関数(演習8-4[1]解答例)

```
#include <stdio.h>
#define N 10
```

1次配列なら要素数は省略可

```
void Scalar(int a[], int k){
    int i;
    for(i=0; i<N; i++) a[i] = k * a[i];
}
```

配列がポインタ渡しになっているのなら、  
「&」を使っても表現可能？

```
int main(void){
    int i;
    int a[N], k=3;
    for(i=0; i<N; i++) a[i] = i;

    Scalar(a, k);

    for(i=0; i<N; i++) printf("a[%d]は%d\n", i, a[i]);
    return 0;
}
```

配列を渡すときは配列名(=同名のポインタ)  
だけで良かった

13

## 2. ポインタと配列

### 正しい！ → &を使って書き換えてみる

```
#include <stdio.h>
#define N 10
```

では、こちらもポインタ変数として  
表現することも可能？

```
void Scalar(int a[], int k){
    int i;
    for(i=0; i<N; i++) a[i] = k * a[i];
}
```

```
int main(void){
    int i;
    int a[N], k=3;
    for(i=0; i<N; i++) a[i] = i;
```

```
    Scalar(&a[0], k);
```

このように変えても同じ結果が得られる

```
    for(i=0; i<N; i++) printf("a[%d]は%d\n", i, a[i]);
    return 0;
}
```

14

## 2. ポインタと配列

それも正しい！ → \*を使って書き換えてみる

```
#include <stdio.h>
#define N 10
```

```
void Scalar(int *a, int k){
    int i;
    for(i=0; i<N; i++) a[i] = k * a[i];
}
```

このように変えても同じ結果が得られる

```
int main(void){
    int i;
    int a[N], k=3;
    for(i=0; i<N; i++) a[i] = i;

    Scalar(&a[0], k);

    for(i=0; i<N; i++) printf("a[%d]は%d\n", i, a[i]);
    return 0;
}
```

書き方が複数あるため、混乱しやすい  
→ 今後も出てくるので覚えておきましょう

15

## 2. ポインタと配列

文字列の入出力 (文字列については、第10回演習と教科書5章参照)

```
int main(void)
{
    char String[20];
    scanf("%19s",String);
    printf("%s\n",String);
    return 0;
}
```

配列名が先頭のアドレスを記録したポインタを表すので'&'をつけなくて良い

printfも%sなら配列名を引数に書く  
(第10回で説明)

関数「scanf」も「%s」を用いて文字列を入力することができる。

(注: 配列の要素数より一つ少ない数を用いて%19sとすることでユーザの入力文字数が宣言した配列の要素数以上になった場合のバッファオーバーランを防ぐことができる)

第7回 変数のアドレスを渡す

```
int main(void){
    int a,b;
    scanf("%d %d", &a,&b);
    printf("a=%d\n b=%d\n",a,b);
    return 0;
}
```

第8回 配列名 (同名のポインタに記録されたアドレス) を渡す

```
int main(void){
    int Account [100];
    .....
    risi(Account,0.003);
    .....
}
```

16



## 2. ポインタと配列

### 文字列の入出力 (文字列については、第10回演習と教科書5章参照)

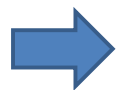
```
int main(void)
{
    char String[20];
    scanf("%19s",String);
    printf("%s\n",String);
    return 0;
}
```

配列名が先頭のアドレスを記録したポインタを表すので「&」をつけなくて良い

関数「scanf」も「%s」を用いて文字列を入力することができる。

(注: 配列の要素数より一つ少ない数を用いて%19sとすることでユーザの入力文字数が宣言した配列の要素数以上になった場合のバッファオーバーランを防ぐことができる)

printfも%sなら配列名を引数に書く (第10回で説明)



```
int main(void)
{
    char String[20];
    scanf("%19s",&String[0]);
    printf("%s\n",&String[0]);
    return 0;
}
```

このように書き換えても同じ結果が得られる  
(∵ 配列の先頭のアドレスを渡せば良いので)

17

## 3. ポインタと配列 (上級者向け)

### 配列はメモリ上に連続して置かれる

```
#include <stdio.h>

int main(void)
{
    int data[5] = {1,2,3,4,5};
    int *p = &data[0];
    int i;
    for(i=0; i<5; i++,p++)
        printf("%d,",*p);
    return 0;
}
```

メモリ上の様子

| アドレス | 変数名     | データ    |
|------|---------|--------|
| 100  | data[0] | 1      |
| 101  | data[1] | 2      |
| 102  | data[2] | 3      |
| 103  | data[3] | 4      |
| 104  | data[4] | 5      |
| 105  | (未使用)   | ?????? |

18

### 3. ポインタと配列（上級者向け）

配列はメモリ上に連続して置かれる

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int data[5] = {1,2,3,4,5};
```

```
    int *p = &data[0];
```

```
    int i;
```

```
    for(i=0; i<5; i++,p++)
```

```
        printf("%d,", *p);
```

```
    return 0;
```

```
}
```

配列の先頭のアドレスを  
ポインタ「p」に代入※

メモリ上の様子

| アドレス | 変数名     | データ |
|------|---------|-----|
| 100  | data[0] | 1   |
| 101  | data[1] | 2   |
| 102  | data[2] | 3   |
| 103  | data[3] | 4   |
| 104  | data[4] | 5   |
| 105  | p       | 100 |

※実行文中の\*pはポインタが差すデータであるが、ここでは宣言と同時にアドレスを初期化していることに注意

19

### 3. ポインタと配列（上級者向け）

配列はメモリ上に連続して置かれる

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int data[5] = {1,2,3,4,5};
```

```
    int *p = &data[0];
```

```
    int i;
```

```
    for(i=0; i<5; i++,p++)
```

```
        printf("%d,", *p);
```

```
    return 0;
```

```
}
```

アドレスを加算しながら、  
5個分の値を表示

メモリ上の様子

| アドレス | 変数名     | データ |
|------|---------|-----|
| 100  | data[0] | 1   |
| 101  | data[1] | 2   |
| 102  | data[2] | 3   |
| 103  | data[3] | 4   |
| 104  | data[4] | 5   |
| 105  | p       | 100 |

ポインタ変数も足し算・引き算ができる。

p++    →    p = p + 1  
pの次のアドレスを参照する

20

### 3. ポインタと配列（上級者向け）

#### 配列はメモリ上に連続して置かれる

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int data[5] = {1,2,3,4,5};
```

```
    int *p = &data[0];
```

```
    int i;
```

```
    for(i=0; i<5; i++,p++)
```

```
        printf("%d,", *p);
```

```
    return 0;
```

```
}
```

アドレスを加算しながら、  
5個分の値を表示

メモリ上の様子

| アドレス | 変数名     | データ |
|------|---------|-----|
| 100  | data[0] | 1   |
| 101  | data[1] | 2   |
| 102  | data[2] | 3   |
| 103  | data[3] | 4   |
| 104  | data[4] | 5   |
| 105  | p       | 105 |

実行結果

1,2,3,4,5,

配列は必ず連続しておくことが  
定められているので、必ずこの  
結果になる

21

### 3. ポインタと配列（上級者向け）

#### 配列の添え字とポインタ

配列の各要素へのアクセスは次のように書く

配列名[要素番号] = 4;

配列「data」の  
3番目の要素



data[3] = 4;

メモリ上の様子

| アドレス | 変数名     | データ |
|------|---------|-----|
| 100  | data[0] | 1   |
| 101  | data[1] | 2   |
| 102  | data[2] | 3   |
| 103  | data[3] | 4   |
| 104  | data[4] | 5   |
| 105  | ???     | ??? |

+  
3

ポインタを使って書くとすれば……

配列「data」の  
3番目の要素



\*(&data[0] + 3) = 4;

若干長くて分かりにくい……

22

### 3. ポインタと配列（上級者向け）

#### 配列の添え字とポインタ

配列の各要素へのアクセスは次のように書く

配列名[要素番号] = 4;

配列「data」の  
3番目の要素  data[3]=4;



ポインタを使って書くとすれば……

配列「data」の  
3番目の要素  \*(data + 3) = 4;  
実はこれでも同じ  
添え字を付けない配列の名前は  
その先頭アドレスを返してくれる

23

### 3. ポインタと配列（上級者向け）

#### 配列の添え字とポインタ

配列の各要素へのアクセスは次のように書く

配列名[要素番号] = 4;

配列「data」の  
3番目の要素  data[3]=4;

配列の添え字は、アドレス計算や「\*」などの記号を付けるのを、人間にとって分かりやすいものに書き換えただけ

配列「data」の  
3番目の要素  \*(data + 3) = 4;  
実はこれでも同じ  
添え字を付けない配列の名前は  
その先頭アドレスを返してくれる

24

### 3. ポインタと配列（上級者向け）

配列の添え字とポインタ → 実際はどう管理されているか

#### int型

```
#include<stdio.h>
int main(void){
    int i, a[10]={0};
    for(i=0; i<10; i++)
        printf("a[%d]のアドレスは %p: %lu¥n", i, &a[i], (unsigned long int)&a[i]);
    return 0;
}
```

1アドレスで1バイト  
管理している場合です。

```
a[0]のアドレスは 0x7fffd33c060: 140737441415264
a[1]のアドレスは 0x7fffd33c064: 140737441415268
a[2]のアドレスは 0x7fffd33c068: 140737441415272
a[3]のアドレスは 0x7fffd33c06c: 140737441415276
a[4]のアドレスは 0x7fffd33c070: 140737441415280
a[5]のアドレスは 0x7fffd33c074: 140737441415284
a[6]のアドレスは 0x7fffd33c078: 140737441415288
a[7]のアドレスは 0x7fffd33c07c: 140737441415292
a[8]のアドレスは 0x7fffd33c080: 140737441415296
a[9]のアドレスは 0x7fffd33c084: 140737441415300
```

4つずつ増えている  
⇒ 1要素を4バイトで管理

25

### 3. ポインタと配列（上級者向け）

配列の添え字とポインタ → 実際はどう管理されているか

#### double型

```
#include<stdio.h>
int main(void){
    int i;
    double a[10]={0};
    for(i=0; i<10; i++)
        printf("a[%d]のアドレスは %p: %lu¥n", i, &a[i], (unsigned long int)&a[i]);
    return 0;
}
```

1アドレスで1バイト  
管理している場合です。

```
a[0]のアドレスは 0x7fffd8a478a0: 140736828045472
a[1]のアドレスは 0x7fffd8a478a8: 140736828045480
a[2]のアドレスは 0x7fffd8a478b0: 140736828045488
a[3]のアドレスは 0x7fffd8a478b8: 140736828045496
a[4]のアドレスは 0x7fffd8a478c0: 140736828045504
a[5]のアドレスは 0x7fffd8a478c8: 140736828045512
a[6]のアドレスは 0x7fffd8a478d0: 140736828045520
a[7]のアドレスは 0x7fffd8a478d8: 140736828045528
a[8]のアドレスは 0x7fffd8a478e0: 140736828045536
a[9]のアドレスは 0x7fffd8a478e8: 140736828045544
```

8つずつ増えている  
⇒ 1要素を8バイトで管理

26

### 3. ポインタと配列（上級者向け）

配列の添え字とポインタ → 実際はどう管理されているか

#### char型

```
#include<stdio.h>
int main(void){
    int i;
    char a[10];
    for(i=0; i<10; i++)
        printf("a[%d]のアドレスは %p: %lu¥n", i, &a[i], (unsigned long int)&a[i]);
    return 0;
}
```

1アドレスで1バイト  
管理している場合です.

```
a[0]のアドレスは 0x7fff52ddf4e: 140737306812238
a[1]のアドレスは 0x7fff52ddf4f: 140737306812239
a[2]のアドレスは 0x7fff52ddf50: 140737306812240
a[3]のアドレスは 0x7fff52ddf51: 140737306812241
a[4]のアドレスは 0x7fff52ddf52: 140737306812242
a[5]のアドレスは 0x7fff52ddf53: 140737306812243
a[6]のアドレスは 0x7fff52ddf54: 140737306812244
a[7]のアドレスは 0x7fff52ddf55: 140737306812245
a[8]のアドレスは 0x7fff52ddf56: 140737306812246
a[9]のアドレスは 0x7fff52ddf57: 140737306812247
```

1つずつ増えている  
⇒ 1要素を1バイトで管理

ここから演習課題です

## 4. 演習課題

### 演習9-1 配列を引数とする関数(&や\*の利用)

```
#include <stdio.h>
#define N 10

void Scalar(int *a, int k){
    int i;
    for(i=0; i<N; i++) a[i] = k * a[i];
}

int main(void){
    int i;
    int a[N], k=3;
    for(i=0; i<N; i++) a[i] = i;

    Scalar(&a[0], k);

    for(i=0; i<N; i++) printf("a[%d]は%d\n", i, a[i]);
    return 0;
}
```

- ・このプログラムをそのまま実行してみよう
- ・赤字部分を書き換えてみよう

29

## 4. 演習課題

### 演習9-2 値渡し？ ポインタ渡し？ 総合問題①

```
#include <stdio.h>
void test(int a, int *b, int c[], int *d, int *e){
    int tmp;
    tmp = a; a = *b; *b = tmp;
    tmp = c[0]; c[0] = c[1]; c[1] = tmp;
    tmp = *d; *d = *e; *e = tmp;
}

int main(void){
    int x = 1, y = 2, z[3] = {3, 4, 5}, *p, *q;
    p = &z[1]; q = &z[2];
    test(x, &y, z, p, q);
    printf("x = %d, y = %d, z[0] = %d, z[1] = %d, z[2] = %d\n",
           x, y, z[0], z[1], z[2]);
    return 0;
}
```

最終的に, x, y, z[0], z[1], z[2]には何が入るでしょうか.  
問題を解いた後に実行してみよう

## 4. 演習課題

### 演習9-3 値渡し？ ポインタ渡し？ 総合問題②

```
#include <stdio.h>
void test(int a, int b[], int c[], int d[], int e[]){
    int tmp;
    tmp = a; a = b[0]; b[0] = tmp;
    tmp = c[2]; c[2] = c[0]; c[0] = tmp;
    tmp = d[1]; d[0] = e[0]; e[0] = tmp;
}
int main(void){
    int x = 1, y = 2, z[3] = {3, 4, 5}, *p, *q;
    p = &z[1]; q = &z[2];
    test(x, &y, z, p, q);
    printf("x = %d, y = %d, z[0] = %d, z[1] = %d, z[2] = %d\n",
           x, y, z[0], z[1], z[2]);
    return 0;
}
```

最終的に, x, y, z[0], z[1], z[2]には何が入るでしょうか.  
問題を解いた後に実行してみよう(test関数が少し変化)

※この課題は2014年度期末試験問題(一部改変)です

31

## 4. 演習課題

### 演習9-4 アドレスの足し算①(空白数カウンタ)

文字列中の空白文字' 'の数を数える関数whcountを完成させよ.  
例えば, "I am a student of TUAT."という文字列に対して, 5を返すものとする.  
また, 空白文字が2文字以上連続している場合, それぞれ1文字として数える.

```
#include <stdio.h>
_____ whcount(char *s){
    int c = _____;
    while(*s != _____){
        if(*s == _____) c++;
        s++;
    }
    return _____;
}
int main(void){
    int count;
    char String[40]="I am a student of TUAT";
    count = whcount(String);
    printf("空白の数は%dです\n", count);
    return 0;
}
```

※この課題は2014年度中間試験問題の一部です. 下線を埋める問題として出題

32



## 4. 演習課題

### 発展9-1 アドレスの足し算②(英単語数カウンタ)

文字列中の英単語の数を数える関数wdcountを完成させよ。  
例えば, "I am a student of TUAT."という文字列に対して, 6を返すものとする。  
ただし, 空白は半角スペース2つ以上でも良い(全角スペースは入らない)

赤下線部  
は演習9-4  
と同じもの  
が入る

```
#include <stdio.h>
_____ wdcount(char *s){
    int c = _____;
    while(*s != _____){
        while(*s == _____) s++;
        if(*s != _____) c++;
        while((*s != ' ') _____ (*s != '¥0')) s++;
    }
    return _____;
}

int main(void){
    int count;
    char String[40]="I am a student of TUAT";
    count = wdcount(String);
    printf("英単語の数は%dです¥n", count);
    return 0;
}
```

※この課題は2014年度中間試験問題の一部です。下線部を埋める問題として出題<sup>33</sup>

本日の説明は以上です。  
演習課題に取り組んでください。